

Chapter 1: Databases and Database Users

Definitions and Concepts:

- **Data:** Known facts that can be recorded and have an implicit meaning. For example, names, telephone numbers, and addresses of people you know.
- **Database:** A collection of related data. It is a highly organized, interrelated, and structured set of data. **Properties:** A database represents some aspect of the real world, also called the **miniworld** or the **universe of discourse (UoD)**. It is a logically coherent collection of data with an inherent meaning and is built for a specific purpose for an intended group of users.
- **Database System:** Comprised of four components: Users, Database Application, Database Management System (DBMS), and the Database.
- **Database Management System (DBMS):** A computerized system that enables users to create and maintain a database. It is a general-purpose software system that facilitates defining, constructing, manipulating, and sharing databases among various users and applications.
- **Metadata:** Data about data. It is the descriptive information about the database, stored by the DBMS in a database catalog or dictionary.
- **Query:** A request sent to the DBMS to retrieve and manipulate data.
- **Transaction:** A program that reads and writes data into the database.

DBMS Functionality

A DBMS provides several key functions:

- **Define a database:** It allows for the definition of a database in terms of its **data types**, **structures**, and **constraints**. The descriptive information, or **meta-data**, is stored in a database catalog or dictionary.
- **Construct or Load:** This involves storing the data on a secondary storage medium.
- **Manipulating the database:** This includes **querying**, generating reports, and performing **insertions**, **deletions**, and **modifications** to the content.
- **Concurrent Processing and Sharing:** The DBMS allows a set of users and programs to share and access data simultaneously while keeping it valid and consistent.
- **Protection:** It provides protection against hardware or software malfunction and unauthorized or malicious access.
- **Maintenance:** The database can be maintained and updated for a long period of time.

Files VS Databases

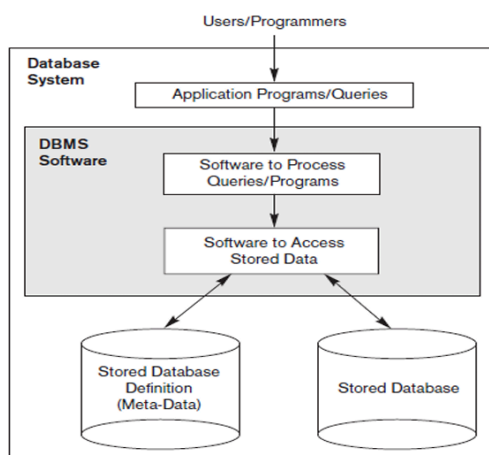
A DBMS offers several advantages over a traditional file system. File systems include **program-data dependence**, where application programs are tied to specific file formats; **lack of security**, as file systems have limited access controls; and **data isolation**, with data being stored in separate files, making it hard to combine. Additionally, file systems can suffer from

data inconsistency, duplication, and inflexibility, making it difficult to change data structures or create new, non-fixed queries.

Characteristics of the Database Approach

- **Self-describing nature of a database system:** A database system contains both the database itself and a complete description of its structure and constraints. This descriptive information, known as **metadata** (data about data), is stored in the **DBMS catalog** or dictionary. The catalog describes the structure of each file, its data types, storage format, and various constraints on the data.
- **Insulation between programs and data, and data abstraction:**
 - **Program-data independence:** This allows the data storage structures and operations to be changed without requiring changes to the DBMS access programs.
 - **Program-operation independence:** User application programs can interact with data by simply invoking operations by name, regardless of how the operations are internally implemented.
 - The combination of these two characteristics is called **data abstraction**, which suppresses the details of data organization and storage to highlight essential features for a better understanding of the data.
- **Support of multiple views of the data:** A database can support different **views** for different types of users. A view is a subset of the database or may contain virtual data that is derived from the database files but is not explicitly stored.
- **Sharing of data and multiuser transaction processing:** The DBMS allows multiple users to access the database concurrently. **Concurrency control** mechanisms ensure that concurrent transactions operate correctly and efficiently, preventing inconsistencies that could arise from multiple users trying to update or retrieve the same data at the same time.

SIMPLIFIED DATABASE SYSTEM ENVIRONMENT



INTERNAL STORAGE FORMAT FOR A STUDENT RECORD, BASED ON THE DATABASE CATALOG

Data Item Name	Starting Position in Record	Length in Characters (bytes)
Name	1	30
Student_number	31	4
Class	35	1
Major	36	4

Figure 1.4
Internal storage format
for a STUDENT record,
based on the database
catalog in Figure 1.3.

EXAMPLE OF A DATABASE CATALOG

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
****	****	****
****	****	****
****	****	****
Prerequisite_number	XXXXNNNN	PREREQUISITE

Figure 1.3
An example of a
database catalog for
the database in
Figure 1.2.

Advantages of using a DBMS:

- Controlling redundancy in data storage and development and update efforts.
- Restricting unauthorized access and providing backup and recovery services.
- Providing multiple user interfaces.
- Representing complex relationships among data.
- Enforcing integrity constraints.
- Permitting inferencing and actions using rules.
- Providing a shared repository of data that can be accessed by many users.
- Enforcing standards and reducing application development time.
- Having the flexibility to change the database structure as requirements change.
- Availability of up-to-date information.
- Economies of scale by combining all operational data.

When NOT to use a DBMS:

- When there are high initial costs for hardware, software, and training.
- When there is a need for overhead for security, concurrency control, recovery, and integrity functions.
- When there is no multiple-user access to data.
- For embedded systems with limited storage capacity.

- For applications with stringent, real-time requirements where DBMS overhead is an issue (e.g., telephone switching systems).
 - For simple, well-defined database applications that are not expected to change.
-

Chapter 2: Database System Concepts and Architecture

Definitions and Concepts:

- **Data Model:** A collection of concepts used to describe the structure of a database, the set of basic operations for specifying retrievals and updates, and certain constraints that the database must fulfill. It provides the necessary means to achieve data abstraction.
- **Database Structure and Operations:**
 - The structure includes data types, relationships, and constraints that apply to the data.
 - **User-defined operations** are specific to a data model, for example, `COMPUTE_GPA` on a `STUDENT` object.
 - **Generic operations** like insert, delete, modify, or retrieve any kind of object are often included in the basic data model operations.
 - **Constraints:** Restrictions that must be followed to ensure valid data.
- **Database Schema:** The description of a database, which is specified during database design and is not expected to change frequently. It includes descriptions of the database structure and the constraints that should hold.
- **Schema Diagram:** An illustrative display of most aspects of a database schema.
- **Schema Construct:** A component or object within the schema, such as `STUDENT` or `COURSE`.
- **Database State (or Snapshot):** The data in the database at a particular moment in time. It is also called the current set of occurrences or instances. The state changes with every insertion, deletion, or update.
 - **Initial Database State:** The database when it is first loaded.
 - **Valid State:** A state that satisfies the structure and constraints.
- **Schema vs. State:** The schema is called **intension**, while the state is called **extension**. While the schema changes very infrequently, changes known as **schema evolution** can occur as application requirements change (e.g., adding a `Date_of_birth` to the `STUDENT` schema).

SCHEMA DIAGRAM FOR THE DATABASE COMPANY RELATIONAL DATABASE SCHEMA

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Schema Construct: Each object in the schema i.e. STUDENT or COURSE

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	-----	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
-------	---------	---------	----------------

DEPT_LOCATIONS

Dnumber	Dlocation
---------	-----------

PROJECT

Pname	Pnumber	Plocation	Dnum
-------	---------	-----------	------

WORKS_ON

Essn	Pno	Hours
------	-----	-------

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
------	----------------	-----	-------	--------------

Categories of Data Models: This classification describes the stages of database design, from initial ideas to physical storage.

- **Conceptual Data Model (High-level):** Provides concepts close to how users perceive data. It is used during the requirement-gathering process. Concepts include entities, attributes, and relationships. The Entity/Relationship (ER) model is a popular example.
- **Implementation or (Representational) Data Models:** Represents the logical part of the database, not the physical structure. They are a foundational theoretical concept whose practical implementation is done in a physical data model and are sometimes called record-based data models.
- **Physical Data Models:** Describe how data is stored as files in the computer, including record formats, orderings, and access paths. **Access Path:** A search structure that makes searching for particular records efficient, such as an index or hashing. An index is an example of an access path that allows direct access to data using a keyword.

The Three-Schema Architecture (ANSI/SPARC)

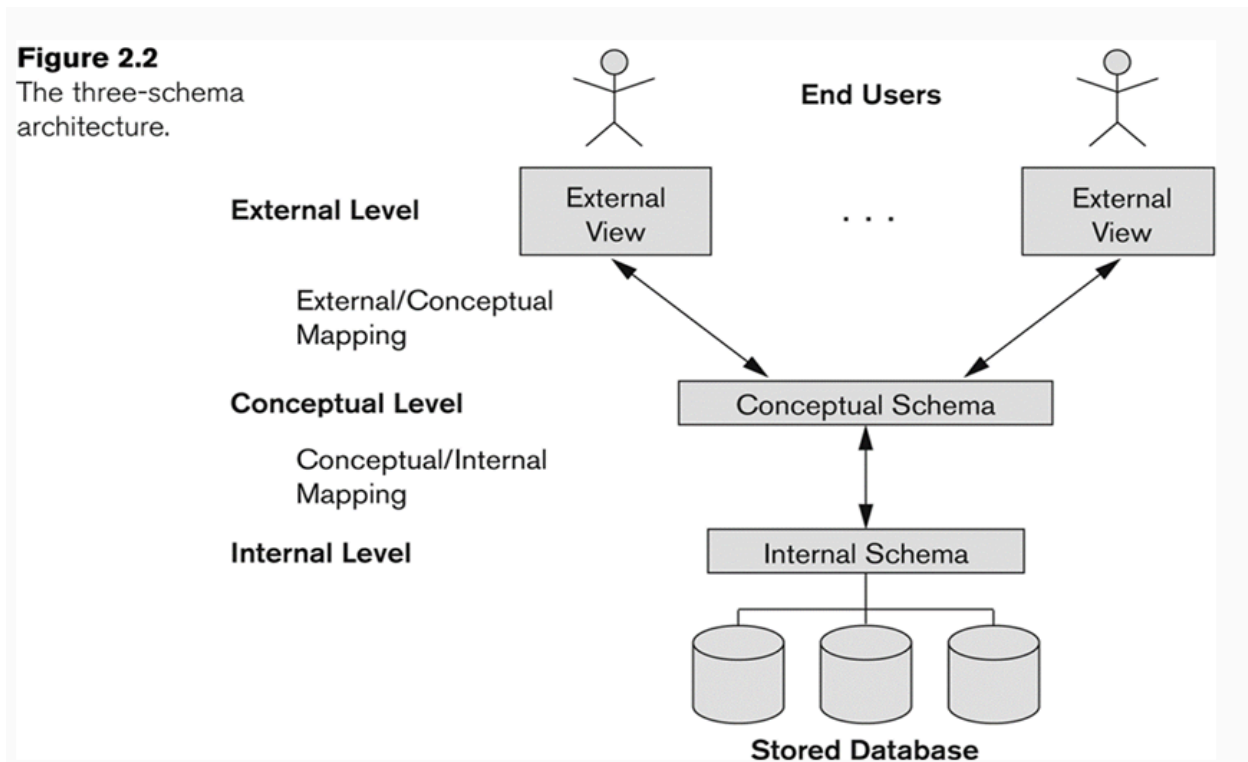
The main goal of this architecture is to separate user applications from the physical database. It supports and visualizes the key DBMS characteristics of being self-describing, having program-data independence, and supporting multiple views of the data.

- **Internal Level:** Describes the physical storage structure. The **Internal schema** details data storage and access paths.
- **Conceptual Level:** Hides physical storage details. It describes entities, data types, relationships, user operations, and constraints. A representational data model is used here.

- **External or View Level:** Consists of multiple **external schemas** or user views. Each external schema describes only the part of the database a particular user group is interested in, hiding the rest.

Figure 2.2

The three-schema architecture.



Data Independence

Data independence is the capacity to change the schema at one level without having to change the schema at the next higher level.

- **Logical data independence:** The capacity to change the conceptual schema without affecting external schemas or application programs.
- **Physical data independence:** The capacity to change the internal schema (e.g., reorganizing files or adding new access paths to improve performance) without changing the conceptual schema. This is easier to achieve than logical data independence.

DBMS Languages

- **Data Definition Language (DDL):** Used by DBAs and designers to define conceptual and internal schemas. Common commands include **CREATE**, **ALTER**, **DROP**, **TRUNCATE**, **COMMENT**, and **RENAME**.
- **Storage Definition Language (SDL):** Used to specify the internal schema when it is separate from the conceptual level.
- **View Definition Language (VDL):** Used to specify user views and their mappings to the conceptual schema. SQL is often used for this purpose.

- **Data Manipulation Language (DML):** Used to manipulate data, including retrieval, insertion, deletion, and modification.
 - **High-level (Nonprocedural) DML:** Specifies *what* data to retrieve. It is set-at-a-time (e.g., SQL) and also called a **Declarative Language**.
 - **Low-level (Procedural) DML:** Specifies *how* to retrieve data, processing one record at a time.
- **Data Control Language (DCL):** Deals with rights and permissions (e.g., **GRANT**, **REVOKE**).
- **Transaction Control Language (TCL):** Manages transactions (e.g., **COMMIT**, **ROLLBACK**, **SAVEPOINT**).

DBMS Interfaces

- **Menu-based Interfaces:** For web clients or browsing.
- **Apps for Mobile Devices:** Allow access to data via mobile phones.
- **Forms-based Interfaces:** Designed for naive users to fill out forms for data entry or retrieval.
- **Graphical User Interfaces (GUIs):** Display a schema diagrammatically, allowing users to manipulate the diagram to specify a query.
- **Natural Language Interfaces:** Accept requests in natural language, interpret them using a schema and dictionary, and generate a high-level query.
- **Keyword-based Database Search:** Similar to web search engines, matching keywords with documents using predefined indexes and ranking functions.
- **Speech Input and Output:** Used for inquiries where a library of predefined words can set up query parameters.
- **Interfaces for Parametric Users:** For users performing repetitive transactions, like bank tellers.
- **Interfaces for the DBA:** Provide privileged commands for managing accounts, granting authorizations, and changing schemas.

The Database System Environment

The database and DBMS catalog are typically stored on disk. The environment includes various user types and internal DBMS modules.

- **Users:** Include DBA staff, casual users, application programmers, and parametric users.
- The **query optimizer** rearranges and reorders operations to improve efficiency.
- A **precompiler** extracts DML commands from application programs.
- The **runtime database processor** executes commands and query plans, working with the system catalog and the **stored data manager** for low-level I/O operations.

THE DATABASE SYSTEM ENVIRONMENT

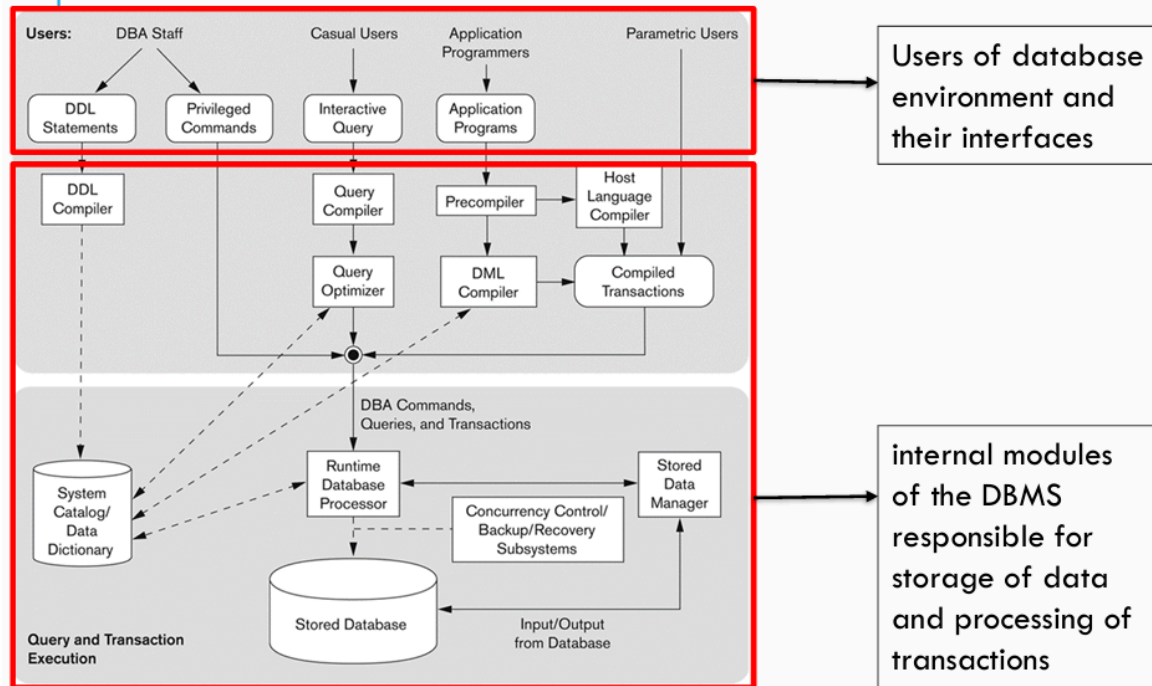


Figure 2.3
Component modules of a DBMS and their interactions.

Database System Utilities

These tools help the DBA manage the system.

- **Loading:** Used to load existing data files into the database.
- **Backup:** Creates a copy of the database, which can be a full dump or an **incremental backup** of only the changes.
- **Database storage reorganization:** Reorganizes files and creates new access paths to improve performance.
- **Performance Monitoring:** Tracks usage and provides statistics for DBA decisions.
- Other utilities include sorting, data compression, and network interfacing.

Tools, Application Environments, and Communications Facilities

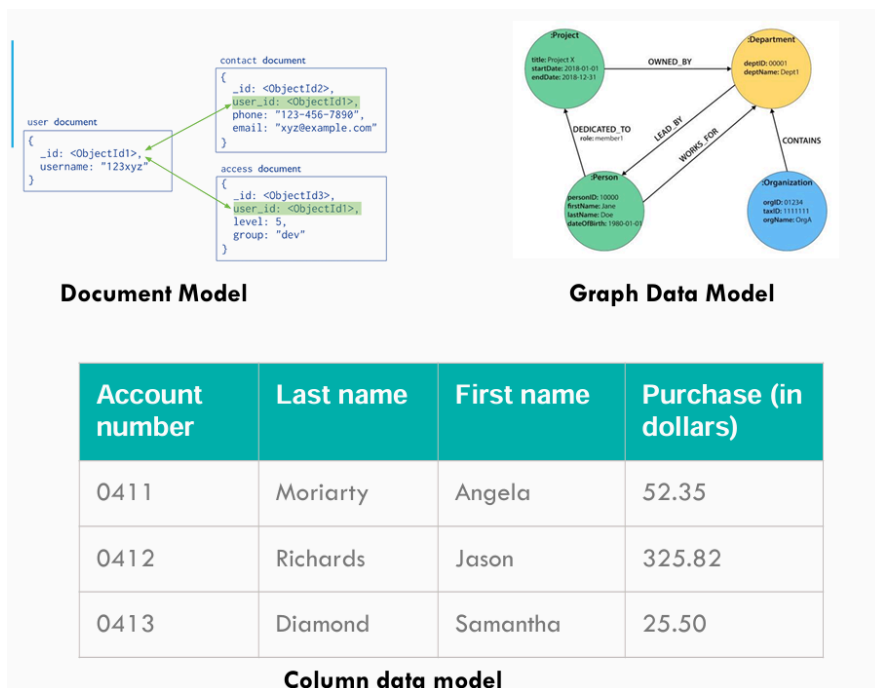
- **CASE Tools:** Used during the design phase of database systems.
- **Data Dictionary (Information Repository):** Stores catalog information about schemas and constraints, as well as design decisions, usage standards, application program descriptions, and user information.
- **Application Development Environment:** Provides tools for database design, GUI development, querying, updating, and application program development. Examples include PowerBuilder and JBuilder.

- **Communications Facilities:** Provided using data communications hardware such as Internet routers, phone lines, long-haul networks, local networks, or satellite communication devices.

Classification of DBMS

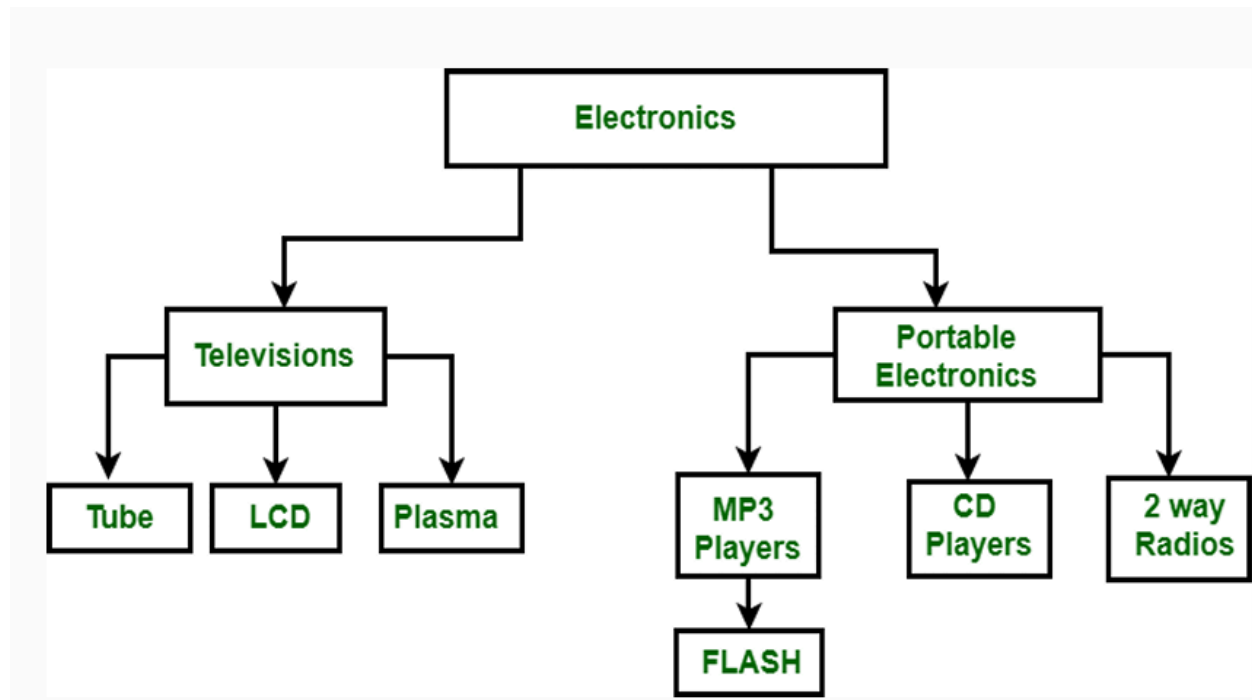
1. Data Model: This classification categorizes actual DBMS software based on the underlying data model it supports and other functional criteria.

- **Relational Data Model (SQL systems):** This is a commercial model where the database is a collection of tables. Each table can be stored as a separate file. It uses SQL and supports a limited form of user views.
- **Object Data Model:** This model defines a database in terms of objects, their properties, and their operations. Objects with the same structure and behavior belong to a class, and classes are organized into hierarchies or acyclic graphs. Operations for each class are specified by predefined procedures called methods.
- **Big Data Systems (NoSQL):** These include:
 - **Key-Value:** Associates a unique key with each value, providing very fast access.
 - **Document Data Model:** Stores data as documents that resemble complex objects, often based on JSON.
 - **Graph Data Model:** Stores objects as graph nodes and relationships as directed graph edges.
 - **Column-based Data Models:** Store columns of rows clustered on disk pages for fast access and allow for multiple versions of the data.

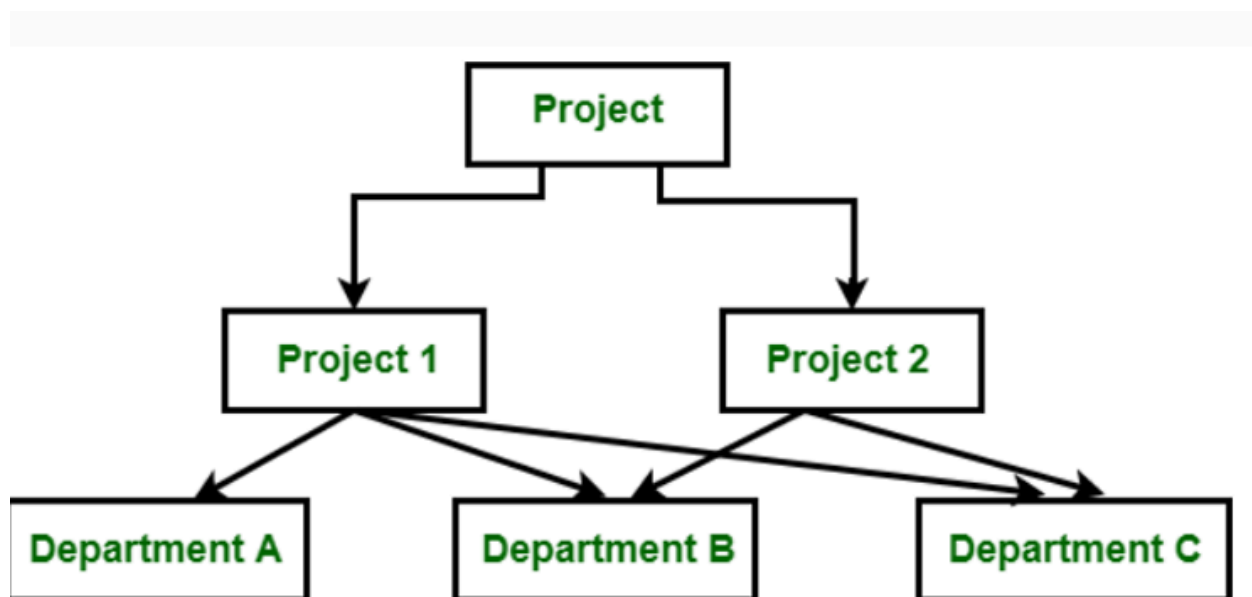


- **Legacy Data Models:**

- **Hierarchical Data Model:** Organizes data in a tree-like structure with nodes connected by branches. Each node has exactly one parent, but a parent may have many children. The topmost node is called the **root node**.



- **Network Data Model:** An advanced version of the hierarchical model where a child can have more than one parent.



- **Tree-structured Data Model:** An XML-based model.

2. Number of Users

- **Single-user systems:** Support only one user at a time.
- **Multiuser systems:** Support multiple users at a time. The majority of databases are multiuser systems.

3. Number of Sites

- **Centralized:** Data is stored at a single computer site and can support multiple users.
- **Distributed DBMS (DDBMS):** The database and DBMS software are distributed over many sites connected by a computer network.
 - **Homogeneous DDBMS:** Uses the same DBMS software at all sites.
 - **Heterogeneous DDBMS:** Uses different DBMS software at each site.

4. Cost

There is often an additional cost for accessing additional features, such as data warehousing, data mining, and maintenance. Some free relational DBMSs include MySQL and PostgreSQL.

5. Purpose

- **General-purpose DBMSs:** Used for a wide range of applications.
- **Special-purpose DBMSs:** Used for a specific application, especially when performance is a primary consideration. They are also known as **Online Transaction Processing (OLTP)** systems, such as an airline reservation system.

Chapter 5: The Relational Data Model and Relational Database Constraints

Relational Model Concepts

The relational model is based on the mathematical concept of a **Relation**. It was first proposed by Dr. E.F. Codd in 1970 and revolutionized database management. Informally, a relation is a **table of values** or a **flat file** of records.

Informal vs. Formal Terms

- **Table** is a relation.
- **Column Header** is an **attribute**.
- **Row** is a **tuple**.
- **Table Definition** is a **schema of a relation**.
- **Populated Table** is the **state of the relation**.

<u>Informal Terms</u>		<u>Formal Terms</u>
Table		Relation
Column Header		Attribute
All possible Column Values		Domain
Row		Tuple
Table Definition		Schema of a Relation
Populated Table		State of the Relation

Formal Definitions

- **Relation Schema $R(A_1, A_2, \dots, A_n)$** : The name of the relation (R) and its attributes (A_1 to A_n). The degree (or arity) of a relation is the number of attributes.
- **Domain**: A set of **atomic** (indivisible) values. An attribute's name defines the role a domain plays in a relation.
- **Tuple**: An ordered set of values $\langle v_1, v_2, \dots, v_n \rangle$, where each value is from the appropriate domain.
- **Relation State $r(R)$** : A specific set of tuples that is a subset of the Cartesian product of the domains of its attributes. It is also called the **relation extension**, while the schema is called the **intension**. The state changes as data is inserted, deleted, or modified, whereas the schema is relatively static.
- **Null Value**: A special value used to represent data that is unknown, not available, or inapplicable.

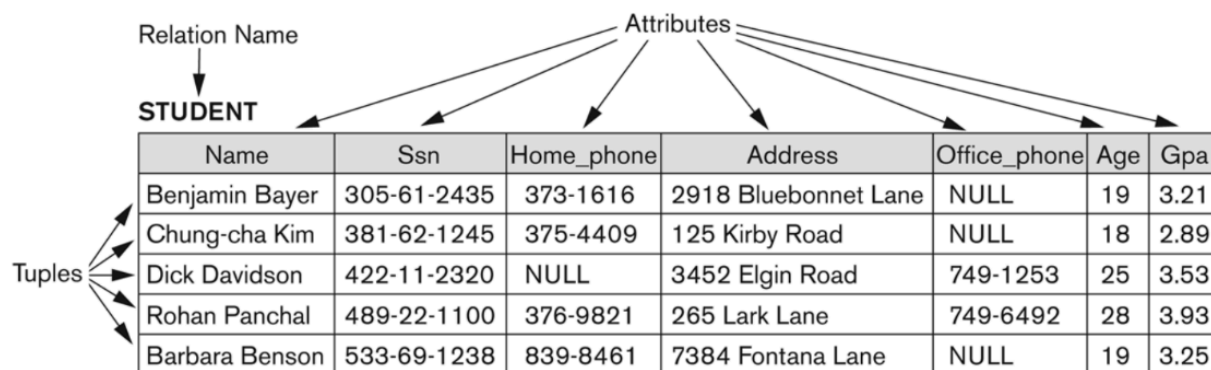


Figure 5.1
The attributes and tuples of a relation STUDENT.

Characteristics of Relations

- **Ordering of tuples:** Tuples are not considered to be ordered, even though they appear so in a table.
- **Ordering of attributes:** The ordering of attributes is not important because each attribute is self-describing; its name is included with its value in a tuple.
- **Atomic Values:** All values in a tuple are considered indivisible, cannot be further divided

$t = \langle (\text{Name}, \text{Dick Davidson}), (\text{Ssn}, 422-11-2320), (\text{Home_phone}, \text{NULL}), (\text{Address}, 3452 \text{ Elgin Road}), (\text{Office_phone}, (817)749-1253), (\text{Age}, 25), (\text{Gpa}, 3.53) \rangle$

$t = \langle (\text{Address}, 3452 \text{ Elgin Road}), (\text{Name}, \text{Dick Davidson}), (\text{Ssn}, 422-11-2320), (\text{Age}, 25), (\text{Office_phone}, (817)749-1253), (\text{Gpa}, 3.53), (\text{Home_phone}, \text{NULL}) \rangle$

Figure 5.3

Two identical tuples when the order of attributes and values is not part of relation definition.

Relational Model Constraints

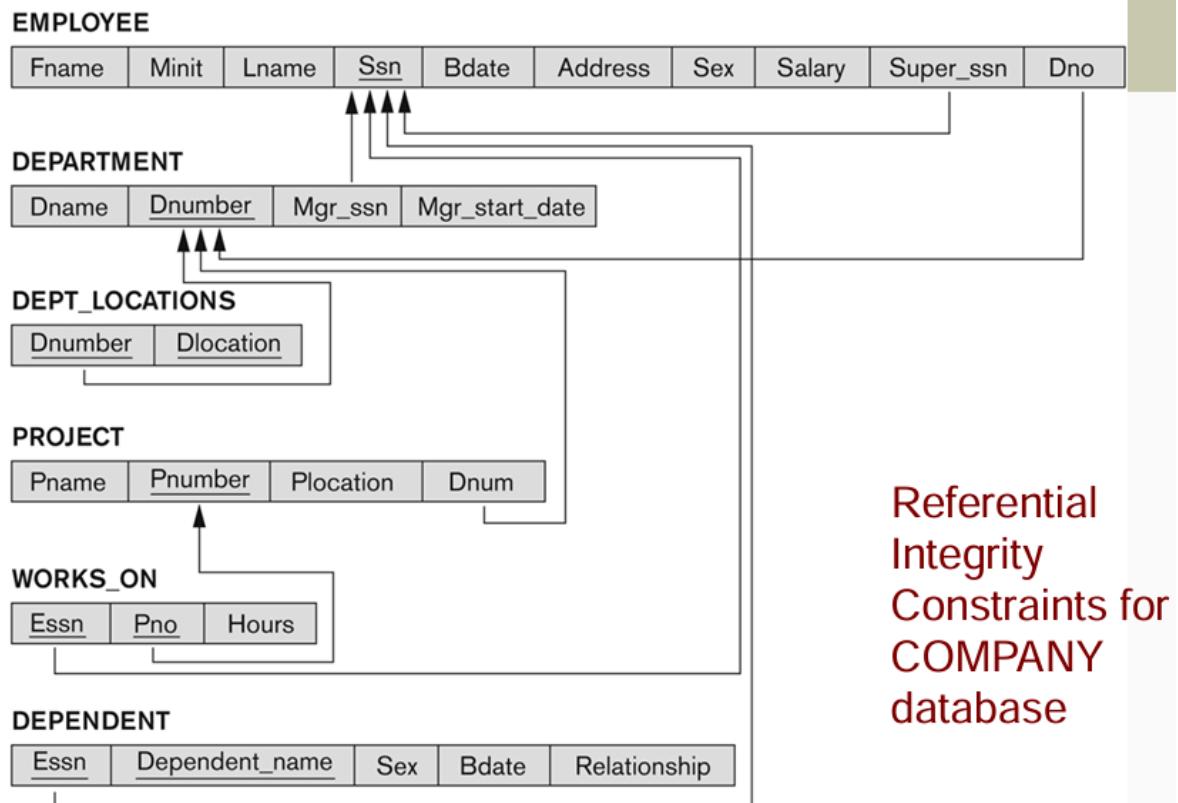
Constraints are conditions that must hold true for all valid relation states.

1. **Inherent or Implicit Constraints:** Based on the data model itself (e.g., a relational model does not allow a list as an attribute value).
2. **Schema-based or Explicit Constraints:** Expressed in the schema.
 - **Domain Constraints:** Each attribute value must be an atomic value from its domain.
 - **Key Constraints:** Used to uniquely identify a tuple.
 - **Superkey:** A set of attributes that uniquely identifies tuples.
 - **Key:** A minimal superkey.

- **NOTE** - Every key is a superkey because it uniquely identifies a tuple, but not every superkey is a key because a superkey may contain redundant attributes that are not necessary for uniqueness.
 - **Candidate Key**: If a relation has multiple keys, each is a candidate key.
 - **Primary Key**: One candidate key is chosen to be the primary key and is underlined in the schema.
- **Entity Integrity Constraint**: The primary key value cannot be **null** because it is used to uniquely identify tuples.
 - **Referential Integrity Constraint**: Specified between two relations (a **referencing relation** and a **referenced relation**). A **foreign** key in the referencing relation must either match a primary key of the referenced relation or be null.
3. **Application-based or Semantic Constraints**: These are business rules beyond the expressive power of the model and must be enforced by application programs.

Figure 5.7

Referential integrity constraints displayed on the COMPANY relational database schema.



Slide 5- 46

Relational Database Schemas and States

- A **Relational Database Schema** is a collection of relation schemas that belong to the same database, along with a set of integrity constraints.

- A **Relational Database State** is a set of relation states that satisfies all integrity constraints. It's also called a database snapshot or instance.

Update Operations and Dealing with Constraint Violations

The three basic update operations are **Insert**, **Delete**, and **Modify**. When an operation violates a constraint, several actions can be taken: **Restrict** (reject the operation), **Cascade** (propagate the changes to other tuples), or **Set Null/Default** (set the foreign key value to null or a default).

- **Insert**: Can violate domain, key, entity integrity, or referential integrity constraints.
- **Delete**: Can only violate referential integrity if the tuple being deleted is referenced by foreign keys.
- **Update**: Updating an attribute that is neither a primary key nor a foreign key usually causes no issues. Updating a primary key is similar to a delete followed by an insert, and updating a foreign key can violate referential integrity.

Transaction

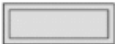
A **transaction** is an executing program with database operations. After its completion, the database must be left in a valid and consistent state that satisfies all constraints.

Chapter 6, 7, 8 SEE SLIDES

Relationship Type	From Entity A → B	From Entity B → A	Example
One-to-One (1:1)	0..1 or 1..1	0..1 or 1..1	Person ↔ Passport
One-to-Many (1:N)	1..1	0..N or 1..N	Teacher → Students
Many-to-One (N:1)	0..N or 1..N	1..1	Students → Teacher
Many-to-Many (M:N)	0..N or 1..N	0..N or 1..N	Students ↔ Courses

Figure 3.14

Summary of the notation for ER diagrams.

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1: N for $E_1:E_2$ in R
	Structural Constraint (min, max) on Participation of E in R

```

CREATE TABLE EMPLOYEE (
    Fname VARCHAR(15) NOT NULL DEFAULT 'Unknown',
    Minit CHAR(1),
    Lname VARCHAR(15) NOT NULL,

    Ssn CHAR(9) NOT NULL PRIMARY KEY,

    Bdate DATE NOT NULL
        CONSTRAINT chk_bdate CHECK (Bdate <= ADD_MONTHS(SYSDATE, -216)),
    -- Or unnamed:
    -- Bdate DATE NOT NULL CHECK (Bdate <= ADD_MONTHS(SYSDATE, -216)),

    Address VARCHAR(30) DEFAULT 'Not Provided',

    Sex CHAR(1)
        CONSTRAINT chk_sex CHECK (Sex IN ('M','F')),
    -- Or unnamed:
    -- Sex CHAR(1) CHECK (Sex IN ('M','F')),

    Salary DECIMAL(10,2) DEFAULT 0
        CONSTRAINT chk_salary CHECK (Salary >= 0),
    -- Or unnamed:
    -- Salary DECIMAL(10,2) DEFAULT 0 CHECK (Salary >= 0),

    Super_ssn CHAR(9),
    Dno INT DEFAULT 1
);

```

```

CREATE TABLE DEPARTMENT (
    Dname VARCHAR(15) NOT NULL UNIQUE,
    Dnumber INT NOT NULL PRIMARY KEY,

    Mgr_ssn CHAR(9) NOT NULL
        CONSTRAINT fk_mgr_name
            FOREIGN KEY REFERENCES EMPLOYEE(Ssn)
            ON DELETE SET NULL
            ON UPDATE CASCADE,
    -- Unnamed version:
    -- Mgr_ssn CHAR(9) NOT NULL
    --     FOREIGN KEY REFERENCES EMPLOYEE(Ssn)
    --     ON DELETE SET NULL
    --     ON UPDATE CASCADE,

    Mgr_start_date DATE NOT NULL DEFAULT SYSDATE
);

```

```

CREATE TABLE DEPARTMENT (
    Dname VARCHAR(15) NOT NULL UNIQUE,
    Dnumber INT NOT NULL PRIMARY KEY,

    Mgr_ssn CHAR(9) NOT NULL
        CONSTRAINT fk_mgr_name
            FOREIGN KEY REFERENCES EMPLOYEE(Ssn)
            ON DELETE SET NULL
            ON UPDATE CASCADE,
    -- Unnamed version:
    -- Mgr_ssn CHAR(9) NOT NULL
    --     FOREIGN KEY REFERENCES EMPLOYEE(Ssn)
    --     ON DELETE SET NULL
    --     ON UPDATE CASCADE,

    Mgr_start_date DATE NOT NULL DEFAULT SYSDATE
);

```

SQL Joins Summary Table

Join Type	What It Does (Precise)	Example Query	Result (Short)
INNER JOIN	Returns only matching rows between both tables.	<pre>SELECT * FROM A INNER JOIN B ON A.id = B.id;</pre>	Only common rows.
LEFT JOIN (Left Outer)	Returns all rows from left table + matching rows from right; unmatched right rows = NULL.	<pre>SELECT * FROM A LEFT JOIN B ON A.id = B.id;</pre>	All A; B only where matches.
RIGHT JOIN (Right Outer)	Returns all rows from right table + matching rows from left; unmatched left rows = NULL.	<pre>SELECT * FROM A RIGHT JOIN B ON A.id = B.id;</pre>	All B; A only where matches.
FULL JOIN (Full Outer)	Returns all rows when there is a match in either table; unmatched rows filled with NULLs.	<pre>SELECT * FROM A FULL OUTER JOIN B ON A.id = B.id;</pre>	All A + all B with NULLs where missing.
CROSS JOIN	Returns Cartesian product: every row in A combined with every row in B.	<pre>SELECT * FROM A CROSS JOIN B;</pre>	A × B combinations.
SELF JOIN	Table joined with itself (use aliases).	<pre>SELECT e1.Name, e2.Name FROM Emp e1 JOIN Emp e2 ON e1.ManagerID = e2.ID;</pre>	Relates rows within same table.
NATURAL JOIN	Automatically joins tables using all columns with the same names.	<pre>SELECT * FROM A NATURAL JOIN B;</pre>	Auto match by same column names.
EQUI JOIN	Join using only = condition. (Same as inner join with equality.)	<pre>SELECT * FROM A JOIN B ON A.id = B.id;</pre>	Matches where values equal.
NON-EQUI JOIN	Join using <, >, BETWEEN, etc, instead of =.	<pre>SELECT * FROM A JOIN B ON A.age BETWEEN B.min AND B.max;</pre>	Match using ranges.

Operation	Symbol	Description	Example (slides style)	Result / Meaning	Notes
Cartesian (Cross) Product	$R \times S$	Combine every tuple of R with every tuple of S . Result has all attributes of R followed by all of S .	<code>EMPNames × DEPENDENT</code>	All possible pairs (one row from <code>EMPNames</code> with one row from <code>DEPENDENT</code>). If τ	R
Cartesian then SELECT (meaningful use)	$\sigma_{\theta} (R \times S)$	Use cross product then filter with SELECT to keep only related combinations.	<code>ACTUAL_DEPS = $\sigma_{(SSN=ESSN)} (EMPNames \times DEPENDENT)$</code>	Keeps only pairs where <code>EMPNames.SSN = DEPENDENT.ESSN</code> — actual employee-dependent pairs.	Equivalent to a join; usual pattern before JOIN existed.
Theta-Join (general join)	$R \bowtie_{\theta} S$	Join using a general condition θ (any comparators: $=, <, >, <=, >=, \neq$ or combinations).	<code>R $\bowtie_{(R.A > S.B)}$ S</code>	Combines tuples from R and S that satisfy θ . Result has attributes of R and S .	General form of join; includes equi-join as special case.
Equi-Join	$R \bowtie_{(R.A = S.B)} S$ (often written $R \bowtie_{(A=B)} S$)	Join where the condition(s) are equality comparisons only.	<code>DEPARTMENT $\bowtie_{(MGRSSN=SSN)}$ EMPLOYEE</code>	Rows where <code>DEPARTMENT.MGRSSN = EMPLOYEE.SSN</code> . Both matching columns typically appear (duplicates).	Equi-join = theta-join restricted to $=$ comparisons.
Natural Join	$R \Join S$ (or $R \bowtie S$ with same-name attrs)	Join on all attributes with same names in both relations and remove duplicate columns in result.	<code>DEPT_LOCS $\Join$ DEPARTMENT $\Join$ DEPT_LOCATIONS</code>	Implicitly uses <code>DEPARTMENT.DNUMBER = DEPT_LOCATIONS.DNUMBER</code> and keeps one <code>DNUMBER</code> column.	If names differ, rename first; natural join hides the join-attribute duplicate.
Inner Join	$R \bowtie S$ (usually with condition)	Same as equi-join (common usage): returns only matching tuples from both relations.	<code>EMPLOYEE $\bowtie_{(DNO = DNUMBER)}$ DEPARTMENT</code>	Only rows where join condition holds (no unmatched rows).	Often "inner join" means equi-join unless θ says otherwise.
Left Outer Join	$R \Join_{\text{left}} S$	Like inner join but keep all tuples from left R ; unmatched S columns are NULL.	<code>EMPLOYEE $\Join_{\text{left}} DEPARTMENT$</code>	All <code>EMPLOYEE</code> rows appear; if no matching <code>DEPARTMENT</code> , department fields = NULL.	Condition is same form as inner join; unmatched left rows preserved.
Right Outer Join	$R \Join_{\text{right}} S$	Like inner join but keep all tuples from right S ; unmatched R columns are NULL.	<code>EMPLOYEE $\Join_{\text{right}} DEPARTMENT$</code>	All <code>DEPARTMENT</code> rows appear; if no matching employee, employee fields = NULL.	Condition same as inner join; unmatched right rows preserved.
Full Outer Join (bonus)	$R \Join_{\text{full}} S$	Keep all tuples from both R and S ; unmatched side's attributes are NULL.	<code>EMPLOYEE $\Join_{\text{full}} DEPARTMENT$</code>	All rows from both sides; matches where condition holds, NULLS where no match.	Some texts denote it differently; included for completeness.

CHAPTER 14

Database Design Theory and Normalization (Simplified Notes)

Normalization means **organizing data properly in a database to:**

1. **Remove redundancy** (store data only once).
2. **Keep logical dependencies** (related data stored together).

It makes databases smaller, faster, and easier to update.

1. Informal Design Guidelines for Relational Databases

Relational database design means grouping attributes to form good tables (relations).

We design mainly the **base relations** (actual stored tables).

A *good relation schema* should follow these four guidelines:

1. Clear meaning of attributes
2. No redundant data
3. Fewer NULLs
4. No spurious (false) tuples when joining tables

1.1 Clear Semantics of Attributes

- Each row (tuple) should represent **one entity or one relationship**.
- Don't mix data of different entities (e.g., EMPLOYEE, DEPARTMENT) in one table.
- Use **foreign keys** to refer to other entities.
- Keep **entity and relationship attributes separate**.
- Design so that each table's meaning can be easily explained.

Guideline 1: Each relation should be easy to explain — no mixed entities or relationships.

Bad design example: Mixing employee and project data in one table causes confusion. Such tables may work as *views* but not as *base tables*.

1.2 Redundant Information and Update Anomalies

If the same data appears in many places, it wastes space and causes **update problems**, called **anomalies**:

Types of Anomalies

1. Insertion anomaly:

- You can't add new data unless related data exists.
- Example: In EMP_PROJ(Emp#, Proj#, Hours, Ename, Pname, Plocation) —
You can't add a new project unless some employee is assigned to it.
You can't add a new employee unless they're assigned to a project.

2. Deletion anomaly:

- Deleting one thing accidentally deletes another.
- Example: If a project is deleted, all employees working on it may also get deleted.

3. Update anomaly:

- Repeated data must be updated in many places.
- Example: If Project P1's name changes, it must be updated in every row (maybe 100+).

Guideline 2: Design tables so **no insertion, deletion, or update anomalies** occur.

1.3 Null Values

NULLs appear when some attributes don't apply to all rows. Too many NULLs waste space and confuse meaning.

Reasons for NULLs:

- Attribute not applicable (e.g., *Visa_Status* for local students)
- Unknown value (e.g., *Date_of_Birth* not recorded yet)
- Unavailable value (e.g., phone number exists but not entered)

Guideline 3: Avoid attributes that are often NULL. If needed, they should be rare and exceptional.

1.4 Spurious Tuples (Fake Results)

When tables are badly designed and joined incorrectly, **false data rows** appear — called **spurious tuples**.

To prevent them:

- Join only on proper **primary–foreign key** pairs.
- Use **lossless join property** — no data should be lost or falsely created after joining.

Guideline 4: Design relations so that joining them using keys does **not create spurious tuples**. Avoid relations that contain matching attributes that are not (foreign key, primary key) combinations because joining on such attributes may produce spurious tuples

Important Properties of Decomposition

When we break a relation (table) into two or more smaller relations, there are **two key properties** we should check:

1. **Non-additive (Lossless) Join Property:**

After decomposition, when we join the smaller relations back together, **we must get exactly the original relation** — no missing data, no extra (spurious) tuples.

→ This ensures no information is lost.

2. **Dependency Preservation Property:**

All the **functional dependencies (FDs)** of the original relation should still hold true in the decomposed relations.

→ This ensures the same data constraints remain valid.

Note:

- Property (a) — *Lossless join* — is **very important** and **must never be sacrificed**.
- Property (b) — *Dependency preservation* — is **less strict** and **can be sacrificed if necessary** (details in Chapter 15).

Functional Dependencies (FDs)

A functional dependency occurs when one attribute uniquely determines another attribute within a relation. A **functional dependency (FD)** shows how attributes relate. If A and B are attributes of relation R, B is functionally dependent on A (denoted $A \rightarrow B$), if each value of A in R is associated with exactly one value of B in R.

If **A determines B**, written $A \rightarrow B$, it means:

- For any two rows, if A values are the same, then B values must also be the same.

Examples:

- $SSN \rightarrow ENAME$
- $PNUMBER \rightarrow \{PNAME, PLOCATION\}$
- $\{SSN, PNUMBER\} \rightarrow HOURS$

FDs are derived from real-world rules (like “each project number has one project name”).

Important: FDs depend on **meaning**, not just data examples. A table instance can show that an FD *does not* hold (if violated), but cannot prove one *must* hold.

Key Terms

- **Determinant:** Attribute(s) on the left side of FD (e.g., A in $A \rightarrow B$)
- **Full functional dependency:**
B fully depends on A, not on any part of A. Example: $\{StaffNo, StaffName\} \rightarrow BranchNo$ is *not full* since $StaffNo \rightarrow BranchNo$ also holds.
- **Partial dependency:**
When part of a key determines another attribute.

Example: In $AC \rightarrow B$ and $A \rightarrow D$ and $D \rightarrow B$ (transitivity), B partially depends on AC.

- **Transitive dependency:**

If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$ (C depends transitively on A).

Normalization of Relations

Normalization = process of organizing data to:

1. Reduce redundancy
2. Avoid anomalies

A **normal form** shows how well a table is organized.

Normal Form	Based on	Main Goal
1NF	Atomic values	No repeating or nested fields
2NF	Keys & FDs	Remove partial dependencies
3NF	Keys & FDs	Remove transitive dependencies
BCNF	Keys & FDs	Stronger version of 3NF
4NF	Multi-valued dependencies	Handle multi-valued facts
5NF	Join dependencies	Handle complex joins

Designers usually normalize **up to 3NF or BCNF** in practice. Sometimes they **denormalize** (combine tables) for performance.

Keys and Attributes

- **Candidate key:** Attribute(s) that uniquely identify a row.
- **Primary key:** One chosen candidate key.
- **Prime attribute:** Part of a candidate key.
- **Non-prime attribute:** Not part of any key.

Decomposition Properties

1. Lossless (Non-additive) Join

When splitting a table into parts:

- The combined attributes of both tables = original attributes. (union)
- Common attributes $\neq$ NULL. (intersection)
- Common attributes must be a key in one of the tables.

Example:

R(A, B, C, D), FD: $A \rightarrow BC$

Decompose into $R_1(ABC)$, $R_2(AD)$.
This is lossless because A is the key for R_1 .

2. Dependency Preservation

All FDs must still be valid in the decomposed tables.

Example:

$R(A, B, C, D)$, FD: $A \rightarrow BC$

Split into $R_1(ABC)$ and $R_2(AD)$ — dependency preserved since $A \rightarrow BC$ is in R_1 .

First Normal Form (1NF)

- Each attribute must have **atomic (indivisible)** values.
- No sets, lists, or nested tables inside a column.

Example:

If EMP_PROJ has a nested “PROJS” column, it must be split into:

- EMP_PROJ1 (Employee info)
- EMP_PROJ2 (Project info)

Second Normal Form (2NF)

A table is in **2NF** if:

- It is already in 1NF, and
- Every **non-key attribute** depends **fully** on the whole primary key (not on part of it).

Example:

$\{SSN, PNUMBER\} \rightarrow HOURS$ (full dependency)

$SSN \rightarrow ENAME$ (partial dependency, violates 2NF)

Third Normal Form (3NF)

A table is in **3NF** if:

- It is already in 2NF, and
- No **transitive dependencies** exist (non-key attributes must not depend on other non-key attributes).

Example:

$SSN \rightarrow DNUMBER$ and $DNUMBER \rightarrow DMGRSSN$

$\rightarrow SSN \rightarrow DMGRSSN$ (transitive dependency) violates 3NF.

Formal Rule:

For every FD $X \rightarrow A$ in R :

- Either X is a superkey, OR
- A is a prime attribute.

Non-Trivial Functional Dependency (Precise)

A functional dependency $X \rightarrow Y$ is **non-trivial** if **Y is not a subset of X**, meaning **Y** contains attributes **not already included** in **X** and therefore provides **new information**.

Example: In the table (*EmployeeID*, *EmployeeName*, *Department*), the dependency **EmployeeID $\rightarrow$ Department** is **non-trivial** because *Department* is not part of *EmployeeID*, and knowing *EmployeeID* gives *new information* about *Department*.

Boyce–Codd Normal Form (BCNF)

BCNF is a **stronger version of 3NF**.

A relation is in BCNF if:

- For every FD $X \rightarrow Y$, **X is a superkey**.

This removes certain anomalies 3NF may miss and ensures better data integrity.

In short: Every determinant must be a superkey.

CHAPTER 20 SEE SLIDES

Property	Quick Check / One-liner
Recoverable (R)	A transaction commits only after all transactions whose changes it read have committed .
Non-recoverable (NR)	A transaction reads uncommitted changes from another transaction that later aborts .
Cascading Rollback (CR)	A transaction reads uncommitted data , so if the first transaction aborts, dependent transactions must roll back too.
Cascadeless (CC)	No transaction reads uncommitted data ; rollbacks do not cascade.
Strict (S)	No transaction reads/writes a data item until the last writing transaction commits . Prevents CR and ensures recoverable.
Conflict Serializable (CS)	Schedule can be rearranged (via swaps of non-conflicting operations) in  serial order; check using precedence/conflict graph (no cycles).

It is important to note that any strict schedule is also cascadeless, and any cascadeless schedule is also recoverable

CHAPTER 21

1. Why Concurrency Control?

- Many transactions run **at the same time**.
- Need to ensure **isolation** $\rightarrow$ no interference.

- Goal: **serializability** (results should look like a serial order).

Main techniques:

1. **Locking (Two-Phase Locking – 2PL)**
2. **Timestamp Ordering**

2. Locks

A **lock** is a variable that controls access to a data item.

The DBMS keeps a **lock table** to track:

- Which item is locked
- What type of lock
- Which transaction(s) hold it

3. Binary Locks (0/1 Locks)

- **Locked (1)** – item cannot be accessed
- **Unlocked (0)** – item can be accessed
- Transaction uses **lock_item(X)** before access and **unlock_item(X)** after use.

Drawbacks:

- Too restrictive → even **reads** require locking → low concurrency.
- Not used in practice.

```

lock_item(X):
B:  if LOCK(X) = 0          (*item is unlocked*)
    then LOCK(X) ← 1      (*lock the item*)
    else
        begin
            wait (until LOCK(X) = 0
                and the lock manager wakes up the transaction);
            go to B
        end;
unlock_item(X):
    LOCK(X) ← 0;          (* unlock the item *)
    if any transactions are waiting
        then wakeup one of the waiting transactions;
  
```

Binary Lock Rules:

- Lock before access
- Unlock after access
- No duplicate locks
- Unlock only if locked

4. Shared/Exclusive Locks (Read/Write Locks)

More flexible than binary locks.

Shared / Read Lock (S-lock)

- Many transactions can read X together.
- No one can write X.

Exclusive / Write Lock (X-lock)

- Only one transaction can read/write X.

read_lock(X):

```
B: if LOCK(X) = "unlocked"
    then begin LOCK(X) ← "read-locked";
        no_of_reads(X) ← 1
    end
else if LOCK(X) = "read-locked"
    then no_of_reads(X) ← no_of_reads(X) + 1
else begin
    wait (until LOCK(X) = "unlocked"
        and the lock manager wakes up the transaction);
    go to B
end;
```

write_lock(X):

```
B: if LOCK(X) = "unlocked"
    then LOCK(X) ← "write-locked"
else begin
    wait (until LOCK(X) = "unlocked"
        and the lock manager wakes up the transaction);
    go to B
end;
```

unlock (X):

```
if LOCK(X) = "write-locked"
    then begin LOCK(X) ← "unlocked";
        wakeup one of the waiting transactions, if any
    end
else if LOCK(X) = "read-locked"
    then begin
        no_of_reads(X) ← no_of_reads(X) - 1;
        if no_of_reads(X) = 0
            then begin LOCK(X) = "unlocked";
                wakeup one of the waiting transactions, if any
            end
    end
end;
```

Locking Generalization

- To **read** X → need S-lock
- To **write** X → need X-lock
- X-lock prevents everyone else from reading/writing
- S-lock prevents writing but allows reading

Shared Lock Rules:

- Read lock before access
- Write lock before access
- Unlock after access
- No duplicate read locks
- No duplicate write locks
- Unlock only if locked

No interleaving:

Once a lock request starts, it must either:

- Be granted, **or**

- Be put into a waiting queue
(before processing other operations)

5. Lock Conversion

A transaction may change the lock it already holds.

Upgrading ($S \rightarrow X$)

- When transaction first reads X but later needs to write X.
- Allowed only if **it is the ONLY reader**.

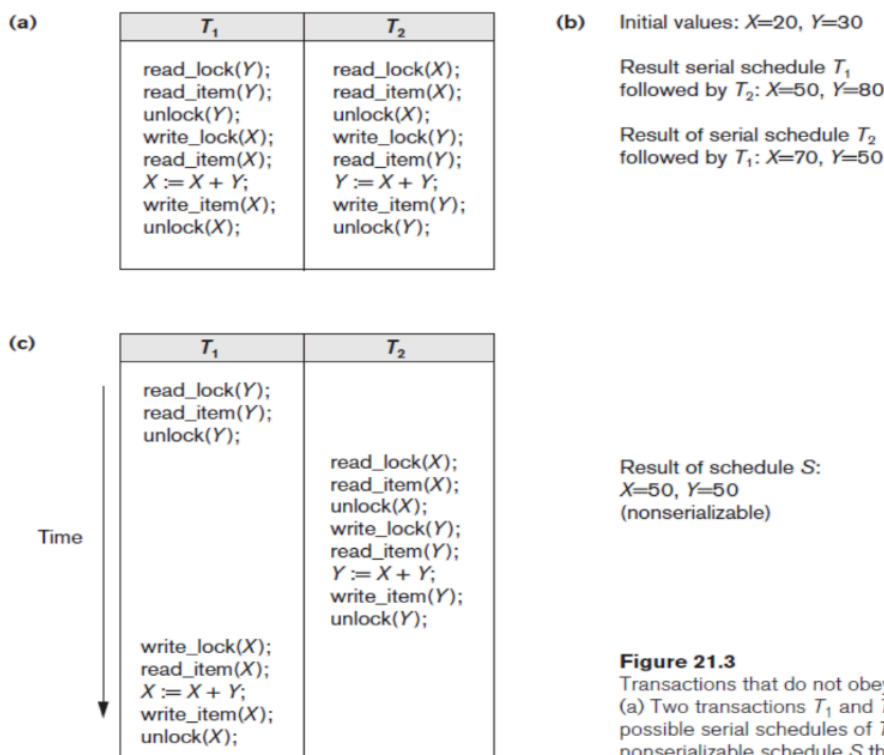
Downgrading ($X \rightarrow S$)

- When it no longer needs to write, only needs to read.

Why store transaction IDs in the lock table?

- To know **which transaction** holds which lock.
- Needed for safe upgrading/downgrading.
- (Also standard practice in locking.)

6. Locking Alone Does NOT Guarantee Serializability



- ✓ Does not guarantee serializability, because in Figure 21.3(a) the items Y in T_1 and X in T_2 were unlocked too early. This allows a schedule such as the one shown in Figure 21.3(c) to occur, which is not a serializable schedule and hence gives incorrect results.

If locks are **released too early**, unsafe schedules occur.

Example from slides:

- T1 unlocks Y too early
 - T2 unlocks X too early
- Nonserializable schedule produced (wrong results).

7. Two-Phase Locking (2PL) — Guarantees Serializability

2PL rule:

All lock operations must occur before the first unlock.

This creates two phases:

(1) Growing Phase

- Can acquire new locks
- Cannot release any
- Upgrades (S→X) happen here

(2) Shrinking Phase

- Can release locks
- Cannot acquire new ones
- Downgrades (X→S) happen here

If EVERY transaction follows 2PL → schedule is always serializable.

Drawback:

- May reduce concurrency, and 2PL will prohibit some serializable schedules

Figure 21.4

Transactions T_1' and T_2' , which are the same as T_1 and T_2 in Figure 21.3 but follow the two-phase locking protocol. Note that they can produce a deadlock.

T_1'	T_2'
<code>read_lock(Y);</code> <code>read_item(Y);</code> <code>write_lock(X);</code> <code>unlock(Y)</code> <code>read_item(X);</code> <code>X := X + Y;</code> <code>write_item(X);</code> <code>unlock(X);</code>	<code>read_lock(X);</code> <code>read_item(X);</code> <code>write_lock(Y);</code> <code>unlock(X)</code> <code>read_item(Y);</code> <code>Y := X + Y;</code> <code>write_item(Y);</code> <code>unlock(Y);</code>

8. Problems With 2PL

2PL does NOT prevent:

- **Deadlock** (two transactions waiting on each other)

	T1	T2	
Granted	write_lock(A)		
	read_item(A)		
		write_lock(B)	Granted
		read_item(B)	
Waiting	write_lock(B)		
	write_item (B)		
		write_lock(A)	Waiting
		write_item (A)	

- **Starvation** (one waits forever)

	T1	T2	T3		
		read_lock(A)			
Wait	write_lock(A)	read_item(A)			
	read_item(A)		read_lock(A)		
	Write_item(A)		read_item(A)	read_lock(A)	
	Unlock(A)	Unlock(A)		read_item(A)	read_lock(A)
			Unlock(A)		read_item(A)
				Unlock(A)	
					Unlock(A)

- **Irrecoverable schedules**

T1	T2	
write_lock(A)		
read_item(A)		
Write_item(A)		
Unlock(A)		
	write_lock(A)	Dirty Read
	read_item(A)	
	Write_item(A)	
	Unlock(A)	
	commit	
Fail		

- **Cascading rollbacks**

T1	T2	T3
write_lock(A)		
read_item(A)		
Write_item(A)		
Unlock(A)		
	write_lock(A)	
	read_item(A)	
	Write_item(A)	
	Unlock(A)	
		write_lock(A)
Fail		read_item(A)
		Write_item(A)
		Unlock(A)

9. Variations of 2PL

A. Basic 2PL

- Standard “two phases” locking
- Does NOT prevent deadlock.

B. Conservative (Static) 2PL

- Lock **all items** before starting.
- Deadlock-free.
- Not practical (must know read-set/write-set in advance).

C. Strict 2PL

- **Write locks are held until commit/abort.**
- Prevents cascading aborts.
- Ensures recoverability.
- Still can deadlock.

Strict 2PL

- Strict-2PL waits until the whole transaction to commit, and then it releases all the locks at a time.
- Strict-2PL protocol does not have shrinking phase of lock release.

Strict 2PL	
T1	T2
s-lock(A)	
read(A)	
x-lock(B)	s-lock(A)
unlock(A)	
read(B)	
write(B)	
	read(A)
commit	unlock(A)
unlock(B)	
	s-lock(B)
	read(B)
	unlock(B)

D. Rigorous 2PL

- Holds **all locks (read + write)** until commit.
- Strongest form.
- Ensures serializability, recoverability, strictness.

Rigorous 2PL	
T1	T2
s-lock(A)	
read(A)	
x-lock(B)	s-lock(A)
	read(A)
read(B)	
write(B)	
commit	
unlock(B)	
	s-lock(B)
	read(B)
unlock(A)	
	commit
	unlock(A)
	unlock(B)

10. Deadlock

Occurs when:

- T1 waits for item locked by T2
- T2 waits for item locked by T1
→ Cycle in wait-for graph.

Transaction does not release any locks (exclusive or shared) until after it commits or aborts

Solutions

1. Deadlock Prevention

- Lock all needed items in advance (impractical)
- Global ordering of items

2. Timestamp-based methods

○ Wait-Die Scheme

- i. Older transaction requests resource held by younger → wait
- ii. Younger transaction requests resource held by older → abort and restart later

○ Wound-Wait Scheme

- i. Older transaction requests resource held by younger → abort younger, restart with same timestamp
- ii. Younger transaction requests resource held by older → wait

Wait – Die	Wound -Wait
It is based on a non-preemptive technique.	It is based on a preemptive technique.
In this, older transactions must wait for the younger one to release its data items.	In this, older transactions never wait for younger transactions.
The number of aborts and rollback is higher in these techniques.	In this, the number of aborts and rollback is lesser.

3. No Waiting (NW):

- Transaction unable to get a lock → immediately aborted and restarted.
- No waiting → no deadlock, but may cause unnecessary aborts.

4. Cautious Waiting (CW):

- Refines wait-die/wound-wait for efficiency and fairness.
- Resource held: If another transaction requests it, check the wait queue.
- Decision: Requesting transaction waits only if queue empty, else aborted → avoids long wait chains.
- Prevents starvation by avoiding indefinite waiting.

5. Deadlock Detection:

- System checks for cycles in the **wait-for graph** → cycle = deadlock.

6. Victim Selection:

- Abort **younger or less-progressed transactions**, avoid long-running ones.

7. Timeouts:

- Transaction waiting beyond **predefined time** → **abort**.

8. Starvation:

- Occurs if a transaction cannot proceed indefinitely.
- **Solution:** First-come-first-served (FCFS) queue.

11. Timestamp-Based Concurrency Control

Timestamp TS(T)

- Unique ID
- Older transaction = smaller timestamp.

Generating Timestamps: Assign unique values using a counter or system clock to enforce a serial order among transactions.

No locks used → No deadlocks.

Each item X stores:

1. **read_TS(X)** = largest timestamp of any transaction that read X
2. **write_TS(X)** = largest timestamp of any transaction that wrote X

Older transactions cannot see or overwrite results of newer ones.

The transaction with the *smaller timestamp* is the OLDER transaction.

The transaction with the *larger timestamp* is the NEWER transaction.

12. Basic Timestamp Ordering (TO) Algorithm

Read(X) by Ti:

- If $\text{write_TS}(X) > \text{TS}(T_i) \rightarrow$ **Ti must abort**
(Ti is too old; it would read a value written “in the future”)
- Else: allow read
Set $\text{read_TS}(X) = \max(\text{read_TS}(X), \text{TS}(T_i))$

Write(X) by Ti:

- If $\text{read_TS}(X) > \text{TS}(T_i) \rightarrow$ abort
- Else if $\text{write_TS}(X) > \text{TS}(T_i) \rightarrow$ abort
- Else $\rightarrow$ do the write
Set $\text{write_TS}(X) = \text{TS}(T_i)$

Issue: may cause starvation (older transaction repeatedly aborts)

13. Strict Timestamp Ordering (Strict TO)

- Ensures schedule is **strict** and **conflict-serializable**.
- Delays a write until all older reads complete.

14. Thomas's Write Rule (Relaxed Version of TO)

Used to **reduce unnecessary aborts**.

For **write(X)** by Ti:

1. If $\text{TS}(T_i) < \text{read_TS}(X) \rightarrow$ abort (Ti tries to write an “old” version)
2. If $\text{TS}(T_i) < \text{write_TS}(X) \rightarrow$ **ignore the write** (obsolete write)
3. Else $\rightarrow$ perform the write

Does NOT guarantee conflict serializability but increases concurrency.

CHAPTER 22

◆ 1. What is Recovery in DBMS?

Recovery = restoring the database to the **most recent consistent state** after a failure.

Two ways to recover:

1. **Restore backup** → used when there is major corruption.
2. **Use the log file** → used when failure is minor and only some changes need undo/redo.

◆ 2. System Log

The log keeps records of:

- transaction start
- read and write operations
- before-image (old value)
- after-image (new value)
- commit or abort

It is used for recovery.

◆ 3. Deferred Update

- Database is **NOT updated until after commit**.
- Only the log is updated before commit.
- After commit → new values are written to database.
- **UNDO not needed**, because DB was never changed.
- **REDO may be needed** to apply committed changes after crash.

Example: (Log For Deferred Update)

- <start_Transaction,T1>
- <read_item,T1,A>
- <write_item,T1,A,10>
- <commit, T1>

10 is the new value

◆ 4. Immediate Update

- Database may be updated **before commit**.
- Log stores both:
 - **old value (BFIM)** → for UNDO
 - **new value (AFIM)** → for REDO
- Supports both UNDO & REDO.

Example: (Log For ImmediateUpdate)

- <start_Transaction,T1>
- <read_item,T1,A>
- <write_item,T1,A,10,11>
- <commit, T1>

10 is the old value

11 is the new value

◆ 5. UNDO & REDO Must Be Idempotent

Meaning:

Doing them multiple times = same effect as doing once.

This avoids errors if recovery restarts after another crash.

UNDO

- used to roll back uncommitted work.
- Makes value go back to the old state.

REDO

- used to reapply updates of committed transactions.
- Makes value go to the new state.

◆ 6. Caching / Buffering in DBMS

Data is stored on disk in **blocks/pages**.

Buffering: DBMS stores frequently accessed disk blocks in a memory buffer pool, managed by algorithms like LRU, MRU, or Clock, to reduce disk I/O.

DBMS Cache: Collection of in-memory buffers tracked by a cache directory.

When used:

- **Read Operation:** DBMS reads from memory if in buffer (cache hit) or loads from disk if not (cache miss).
- **Write Operation:** Updates are made in memory, marked dirty, and written to disk either asynchronously (write-back) or immediately (write-through).

Important concepts:

- **Dirty bit** → 1 if modified, must be written to disk before replacement
- **Pin bit** → prevents DBMS from writing a page back yet
- **Buffer replacement algorithms:** LRU, MRU, Clock

◆ 7. Flushing to Disk: Two Strategies

✓ 1. In-place Update

- Updates overwrite the same location on disk.
- Most common approach.
- Needs logging (UNDO + REDO).

✓ 2. Shadowing

- Keep two copies: old & new.
- Write updated version to a **new location**.
- Not widely used.

Before-image = old value

After-image = new value

◆ 8. Write-Ahead Logging (WAL)

Rule: Write log to disk BEFORE writing data to disk. Record the before-image (BFIM) in the log before overwriting it on disk to allow recovery.

Two conditions:

1. **Before overwriting old value on disk,**
→ write UNDO log record to disk.
2. **Before commit completes,**
→ write all UNDO + REDO log entries to disk.

WAL ensures recovery is always possible.

◆ 9. Steal / No-Steal & Force / No-Force

These describe when updated pages may be written to disk.

✓ Steal

- DBMS is allowed to write pages of an uncommitted transaction.
- This requires **UNDO** during recovery.

✓ No-Steal

- Cannot write uncommitted pages to disk.
- Easier recovery but requires large memory.

✓ Force

- At commit time, DBMS forces all updated pages of the transaction to disk.

✓ **No-Force**

- Does NOT force pages to disk at commit.
- Requires **REDO** during recovery.

Most DBMS use:

👉 **Steal + No-force**: “Steal” lets uncommitted pages be written to disk to save buffer space; “No-Force” allows committed pages to stay in memory, avoiding repeated disk writes and reads.

Aspect	Steal	No-Steal
Dirty Pages	Can be written to disk before commit	Written to disk only after commit
Recovery Complexity	Higher (requires UNDO logging)	Lower (no need to undo uncommitted changes)
Memory Usage	More flexible; frees buffer space as needed	Less flexible; may require large memory
Performance Impact	Better for large transactions	Better for small, quick transactions

Transaction Lists for Recovery: DBMS maintains lists of active, committed, and aborted transactions to make recovery more efficient.

◆ **10. Checkpoints**

A checkpoint = a point where DBMS flushes all data + log records to disk.

Purpose:

- Reduce recovery time
- Ensure committed transactions are safely stored, data integrity is maintained
- Improve system performance

Checkpoint steps:

1. Stop new operations briefly
2. Write all dirty buffers to disk
3. Write checkpoint record to log, and force-write log to disk
4. Resume normal operations

DBMS recovery manager sets intervals based on time or number of committed transactions for checkpointing

◆ **11. Fuzzy Checkpointing**

- Database **does not stop** transactions.
- System logs a checkpoint while continuing work.
- Recovery may need undo/redo depending on transaction state.

Example:

Transaction	Started	Committed	Action
Ta	before checkpoint	before	nothing

Tb	before checkpoint	before failure	redo
Tc	after checkpoint	before failure	redo
Td	after checkpoint	not committed	undo

◆ 12. Transaction Rollback

Needed when:

- a transaction updates data and then fails before commit.

Rollback uses **UNDO** to restore old values.

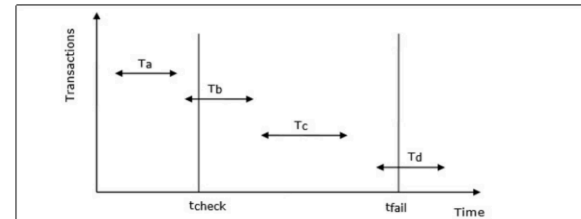
Cascading Rollback

If T1 writes X

- T2 reads X
- T1 aborts
- T2 must also abort
- any transaction reading from T2 must abort
- and so on.

This happens only when schedules are **recoverable but not strict or cascadeless**.

Modern systems avoid cascading rollbacks.



Transaction Rollback: On failure before commit, undo log restores old values; cascading rollbacks affect dependent transactions unless strict (no transaction reads/writes uncommitted data) or cascadeless (no transaction reads uncommitted data) schedules are enforced.

◆ 13. Transactions That Do Not Affect the Database

Examples:

- printing reports
- sending messages

These should only happen **after commit**.

If the transaction fails → these actions must be cancelled.